



MIDDLE EAST TECHNICAL UNIVERSITY

# MMI714

# Generative Models for Multimedia

## The Latent Space

# Welcome

This is MMI714

“Generative Models for Multimedia”

This week we will talk about:

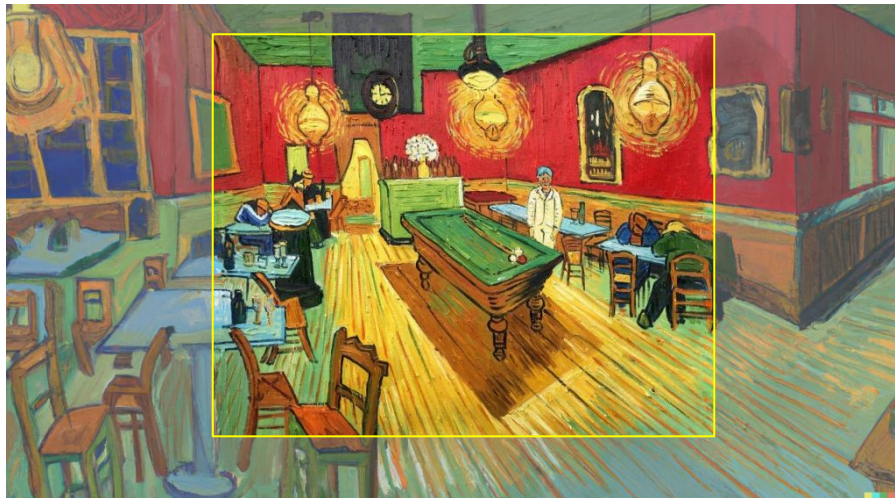
Dimensionality, GMMs,

Feature Spaces, Deep Features,

The Latent Space,

Auto-Encoding, VAEs

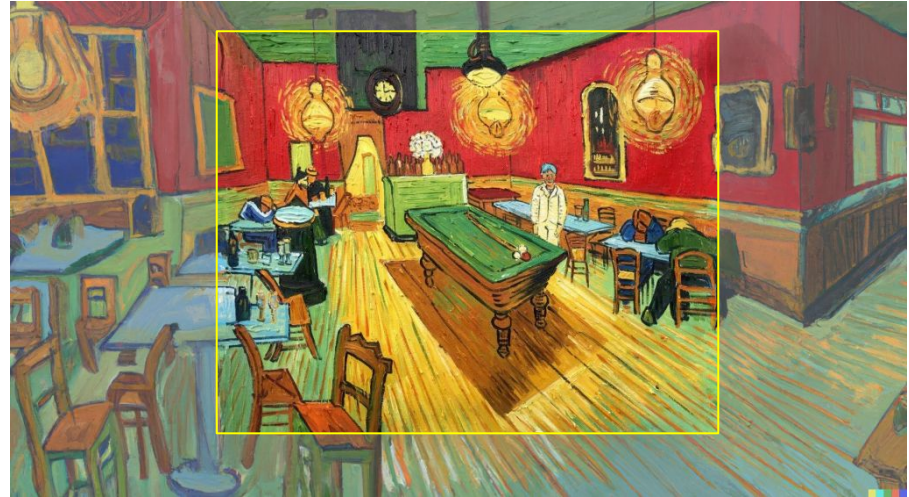
Disentanglement



# Dimensionality

In the context of generative models (and deep learning), **dimensionality** refers to the number of input features or variables that describe the input.

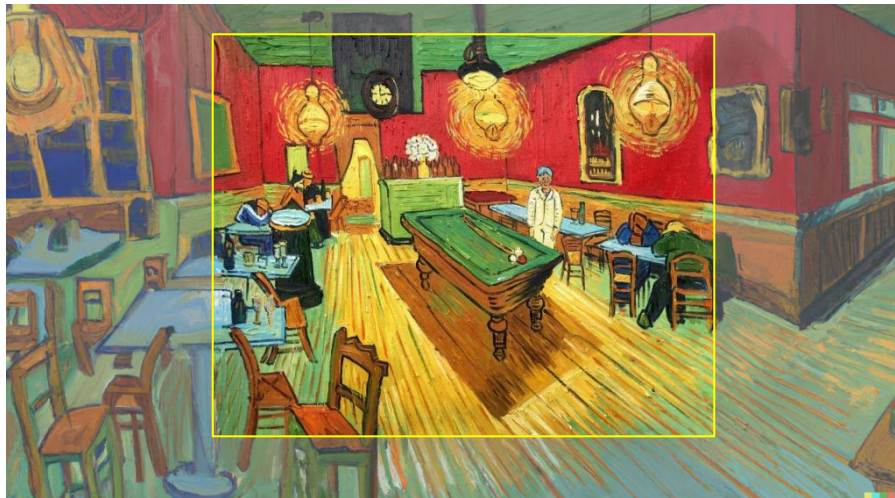
High-dimensional data poses challenges such as increased computational complexity and difficulty in understanding and visualizing the data.



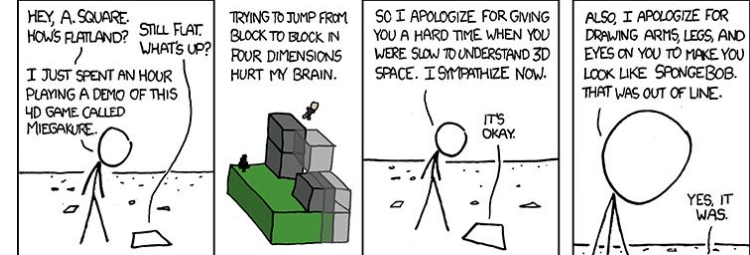
# Dimensionality

**Dimensionality** reduction techniques play a vital role in mitigating these challenges by reducing the number of dimensions while preserving important information.

We'll dive into the motivations, techniques, and benefits of dimensionality reduction in generative modeling. This will lead us to the idea of a "latent space".



# Curse of Dimensionality

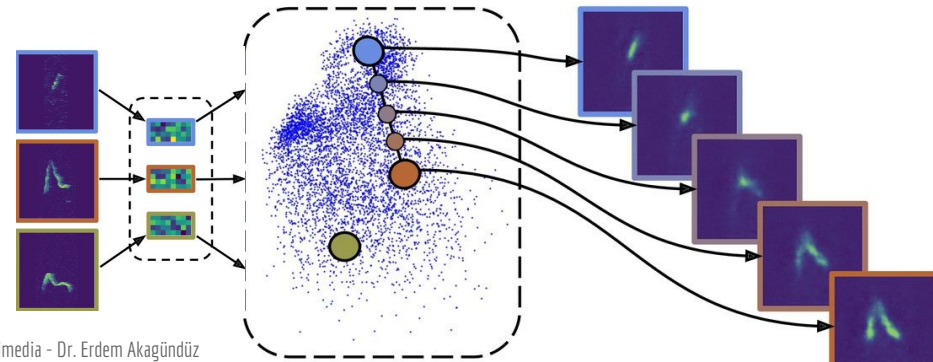


- In generative modeling (and in machine learning general), the curse of dimensionality refers to the challenges that arise when working with high-dimensional data.
- High-dimensional data refers to datasets with a large number of features or variables that describe each data point.
- CoD causes many problems in generative modelling such as “mode collapse” in GANs (which we’ll learn later)
- The curse of dimensionality necessitates the need for dimensionality reduction techniques to address these challenges.

# Sampling from a Vector Space

- In generative modeling, we require a intermediate vector space that is both **continuous** and **representative** of the underlying data distribution for effective sampling from a generator.
- A continuous vector space enables smooth interpolation and exploration of the data, allowing for **seamless transitions** and generation of new samples.

figure from <https://doi.org/10.7554/eLife.67855>

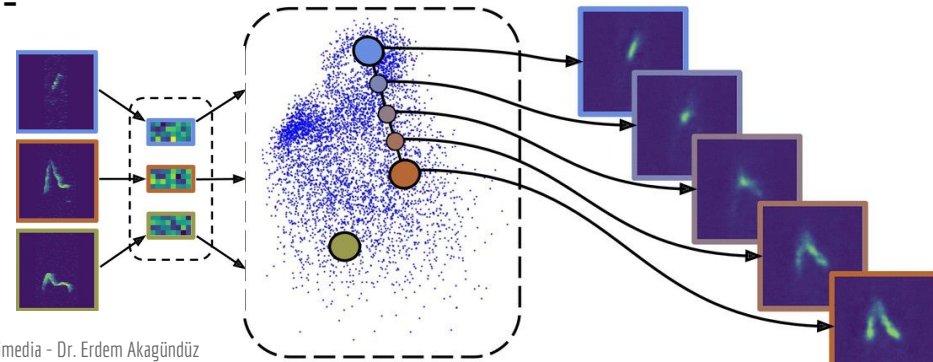




# Sampling from a Vector Space

- Representativeness implies that similar samples or concepts in the original data space should be close to each other in the vector space, facilitating accurate modeling and sampling.
- However, directly working with the high-dimensional data space poses challenges in achieving a continuous and representative vector space due to the curse of dimensionality.
- We need techniques that maps high-dimensional data to a lower-dimensional vector space.

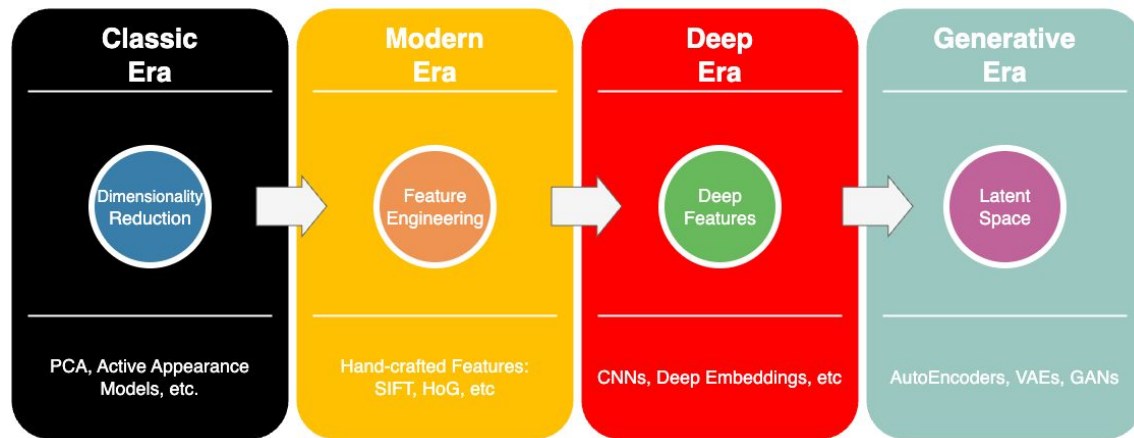
figure from <https://doi.org/10.7554/eLife.67855>





# Evolution

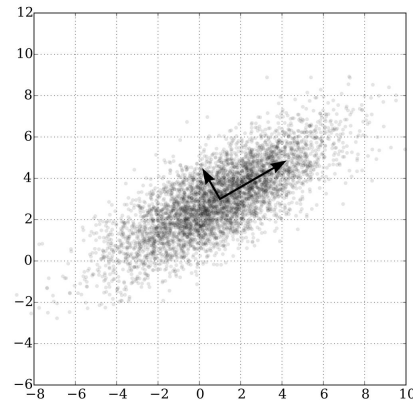
- In the field of generative modeling, the quest to find a suitable vector space for sampling has evolved through distinct eras, each characterized by different techniques and approaches.





# Classical Methods

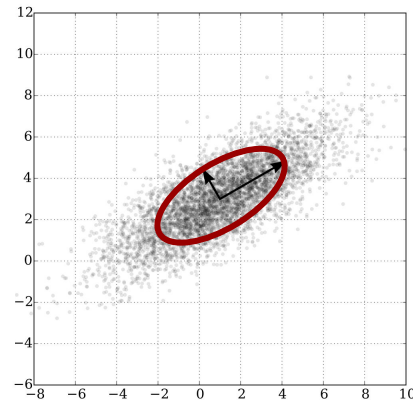
- Classical methods like Principal Component Analysis (PCA) were primarily developed for dimensionality reduction.
- The goal was to capture the most important components of the data, reducing the dimensionality while preserving as much information as possible.
- The resulting principal components capture the directions of maximum variance in the **original data**.



# Classical Methods

- Classical methods like Principal Component Analysis (PCA) were primarily developed for dimensionality reduction.
- The goal was to capture the most important components of the data, reducing the dimensionality while preserving as much information as possible.
- To create a distribution representing the dataset, you can model the principal components as a multivariate normal distribution.

$$p(\mathbf{x}) = \frac{1}{|\sqrt{2\pi}\boldsymbol{\Sigma}|} e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})}$$



# Generation with Classical Methods

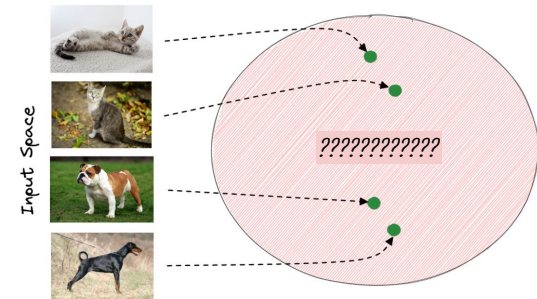
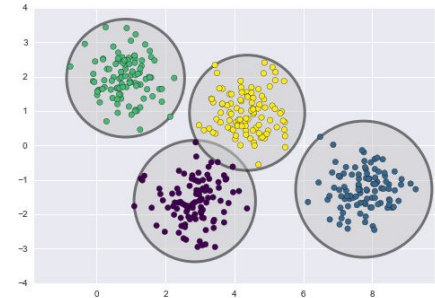
- For complex data (like images etc) this idea failed miserably because:



What if the data is multi-modal? PCA assumes that the data follows a unimodal Gaussian distribution along the principal components



Classical methods may not capture all the complex and high-level semantics of the data. They focus on capturing statistical variations rather than semantic meaning. The generated samples may resemble the statistical properties of the original data, but they may not capture the full complexity or exhibit higher-level semantics.

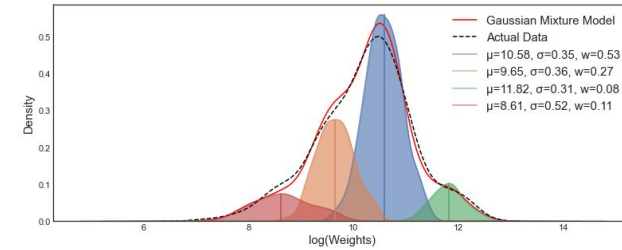


# Gaussian Mixture Models (GMM)



A Gaussian mixture model (GMM) is a generative probabilistic model that consists of multiple Gaussian distributions.

- It is used to model complex data that cannot be represented by a single Gaussian distribution.
- GMMs are commonly used in clustering and density estimation tasks.
- The parameters of a GMM include the number of mixture components, the mean and standard deviation of each component, and the mixing coefficients that determine the weight of each component.

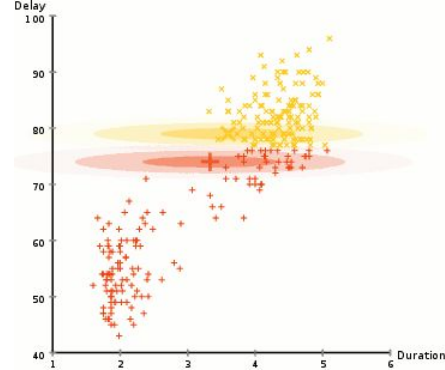


# Expectation-Maximization Algo.

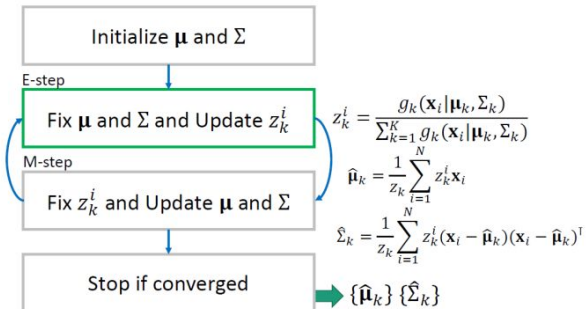


Expectation Maximization (EM) is an iterative algorithm used to estimate the parameters of a statistical model when some of the variables are unobserved.

- EM is commonly used to train Gaussian Mixture Models (GMMs), which are generative models that approximate a probability distribution as a weighted sum of Gaussian distributions.
- In GMMs, each Gaussian distribution represents a cluster in the data, and the weights represent the relative importance of each cluster.
- EM is a powerful and widely used algorithm for unsupervised learning and can be applied to many other types of models beyond GMMs.



EM Algorithm for GMM



# GMM for Generation?



What kinds of problems focus on utilizing simple generation techniques like GMMs for data-related tasks ?

(rather than explicitly incorporating semantic or higher-level feature spaces)

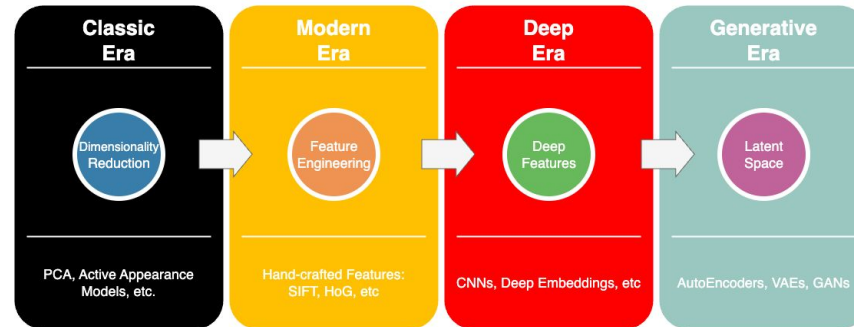
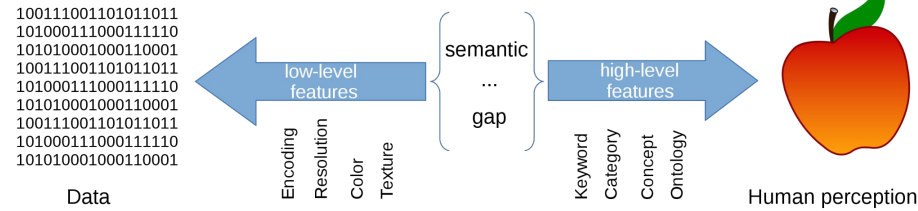
- Simulations
- Games/Animation
- Outlier Detection/Statistical Analysis
- Compression

# Semantic Gap



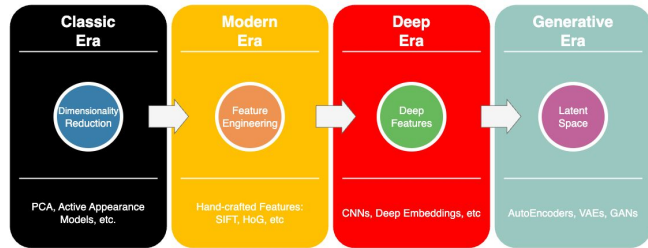
The "semantic gap" is a well-known problem in the field of AI that refers to the disparity between the low-level, perceptual representation of data and the high-level, semantic understanding and interpretation of that data.

- In simpler terms, the semantic gap problem highlights the difficulty in capturing and representing the rich and complex semantics, meaning, and context of data using computational methods.



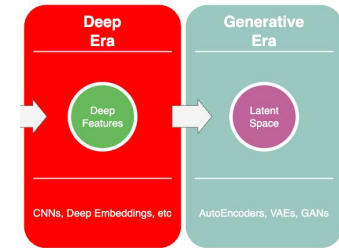


# Feature Space Evaluation



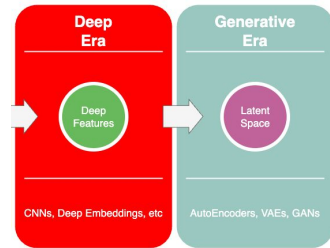
- Modern Era: Pattern Recognition and Handcrafted Features
  - Focus on the use of handcrafted features, such as SIFT and HOG,
  - Dimensionality Reduction → Feature Engineering:
    - (Active Appearance Models)
- Deep Era: Deep Feature Learning
  - By training a deep encoder on a large-scale dataset, automatically extracting hierarchical and abstract features from the input data.
  - Feature Engineering → Feature Learning:
    - (Alexnet)
- Generative Era: The Latent Space
  - Deep features shift into latent spaces for generative purposes
  - Feature Learning → Latent Space:
    - (VAEs)

# Deep Features vs Latent Space



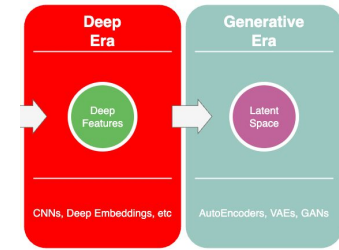
- The difference between deep features learned by a deep encoder and **a latent space of a generative model** lies in their origin, representation, and usage within generative modeling.
- Let's delve into a comparative overview of deep features learned by an encoder and the latent space of a generative model, highlighting their differences and the significance of these distinctions in the context of generative modeling:

# Deep Features vs Latent Space



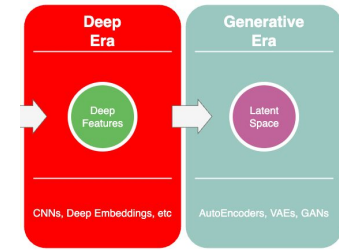
- Representation vs. **Distribution Modelling**
  - **Deep Features:** Deep features learned by an encoder are representations of data that capture high-level patterns and discriminative information. **They focus on encoding the salient features necessary for classification or other downstream tasks.**
  - **Latent Space:** The latent space of a generative model is a lower-dimensional representation of the data that aims to capture the underlying structure and variations. **It focuses on modeling the probability distribution of the data in the latent space for generation purposes.**
- While deep features focus on capturing discriminative information for specific tasks, the latent space in generative models aims to learn a compressed, structured representation that can be used for data synthesis and generation.

# Deep Features vs Latent Space



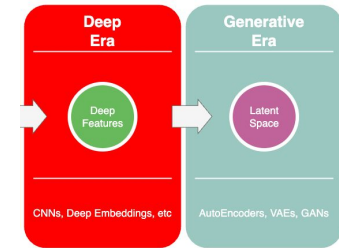
- Supervised vs. **Unsupervised Learning**:
  - **Deep Features**: Learning deep features typically requires labeled data for supervision. **Models are trained using labeled examples to optimize the features for the specific task at hand, such as image classification.**
  - **Latent Space**: Generative models learn the latent space in an unsupervised manner, often using unlabeled data. **The models capture the underlying patterns and variations present in the data distribution without relying on explicit labels.**
- Unsupervised learning of the latent space allows generative models to capture the inherent structure of the data without the need for labeled examples.

# Deep Features vs Latent Space



- Discrimination vs. **Generation**:
  - **Deep Features**: Deep features focus on capturing discriminative information to distinguish between different classes or categories in the data. **They emphasize what differentiates one data point from another.**
  - **Latent Space**: The latent space of a generative model emphasizes the generation of new, realistic samples that resemble the original data distribution. **It encapsulates the factors of variation present in the data, allowing for the synthesis of diverse and novel instances.**
- The focus on generation in the latent space enables generative models to go beyond discrimination and create new data samples. This is crucial for tasks like image synthesis, data augmentation, and generating novel instances in various creative applications.

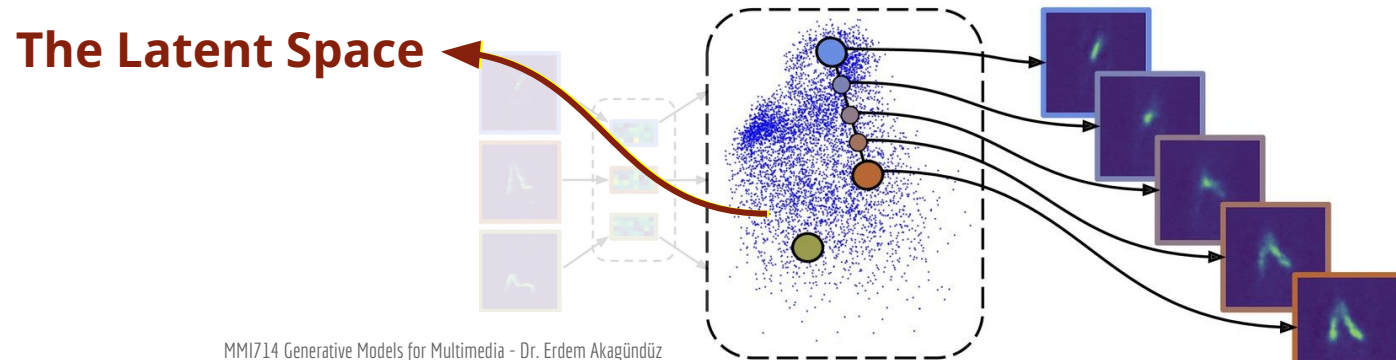
# Deep Features vs Latent Space



- Fixed vs. **Controllable Representations**:
  - **Deep Features**: Deep features learned by an encoder are fixed representations that do not offer explicit control over specific attributes or characteristics of the data. **They are optimized for the specific task and lack explicit manipulability.**
  - **Latent Space**: The latent space of a generative model offers the potential for explicit control and manipulation of specific attributes or factors of variation. **By exploring different regions or interpolating between latent vectors, specific attributes can be modified or combined to generate desired outputs.**
- The controllability of the latent space provides flexibility in generative modeling, allowing users to manipulate specific attributes or create variations in the generated outputs, particularly useful in applications like image editing, style transfer, and interactive content creation.

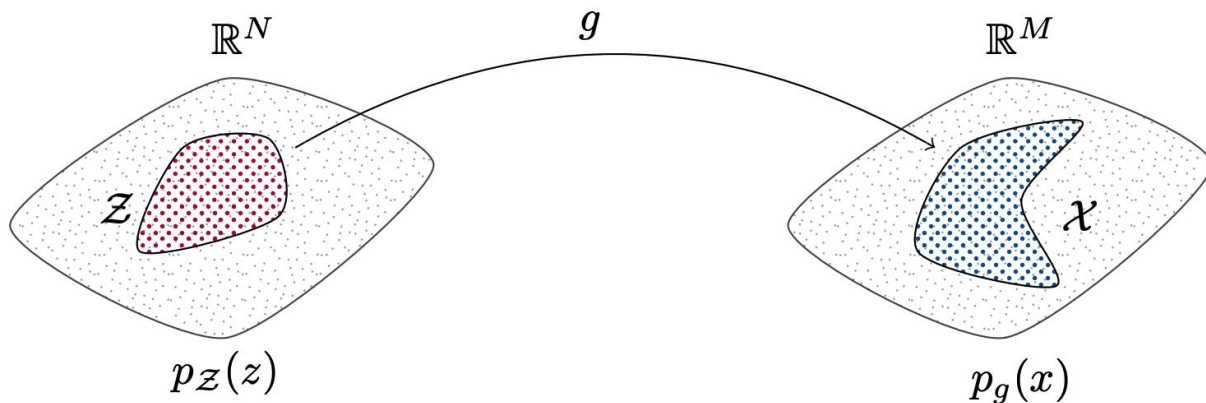
# Sampling from a Vector Space

- In generative modeling, we require a intermediate vector space that is both **continuous** and **representative of the underlying data distribution** for effective sampling from a generator.
- A continuous vector space enables smooth interpolation and exploration of the data, allowing for **seamless transitions** and generation of new samples.



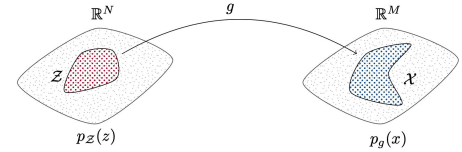
# The Latent Space

- A generative model  $g$  is a function that maps
  - a latent space  $\mathbf{z} \subseteq \mathbb{R}^N$   $\rightarrow$  on a target space  $\mathcal{X} \subseteq \mathbb{R}^M$

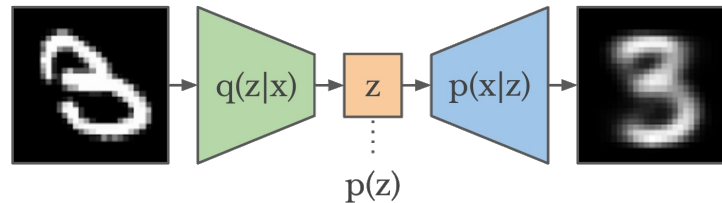




# The Latent Space

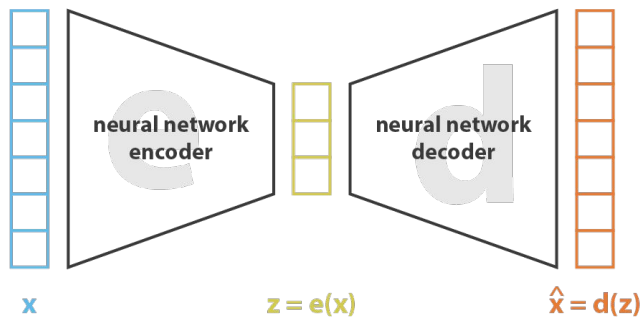


- Early generative models primarily focused on explicit feature engineering and handcrafted representations.
- Latent space was not explicitly defined, and generation relied on manually designed algorithms or statistical models.
- Autoencoders, (not generative models but will evolve in to generative VAEs), played a significant role in developing the concept of the latent space.



# AutoEncoders

- Autoencoders comprise an encoder network that maps input data to a latent representation and a decoder network that reconstructs the input from the latent space.



## Autoencoders

A diagram of the process described by the story is shown in Figure 3-2. You play the part of the **encoder**, moving each item of clothing to a location in the wardrobe. This process is called **encoding**. Brian plays the part of the **decoder**, taking a location in the wardrobe and attempting to re-stitch the item. This process is called **decoding**.

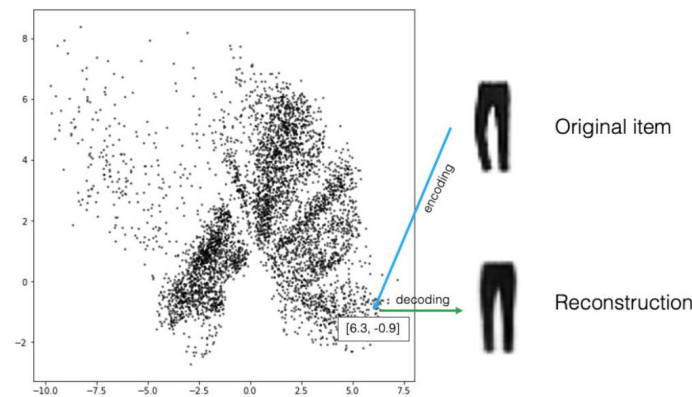


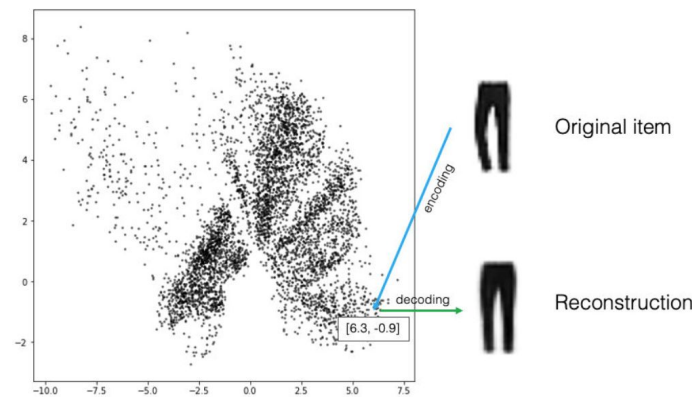
Figure 3-2. Items of clothing in the infinite wardrobe - each black dot represents an item of clothing.

# AutoEncoders

- **Encoder:** A module that compresses the train-validate-test set input data into an encoded representation that is typically several orders of magnitude smaller than the input data.
- **Bottleneck:** A module that contains the compressed knowledge representations and is therefore the most important part of the network.
- **Decoder:** A module that helps the network “decompress” the knowledge representations and reconstructs the data back from its encoded form. The output is then compared with a ground truth.

## Autoencoders

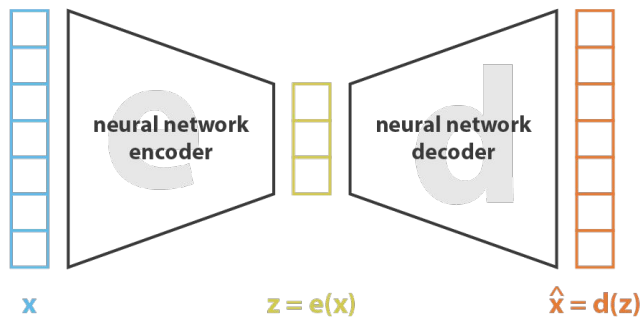
A diagram of the process described by the story is shown in **Figure 3-2**. You play the part of the **encoder**, moving each item of clothing to a location in the wardrobe. This process is called **encoding**. Brian plays the part of the **decoder**, taking a location in the wardrobe and attempting to re-stitch the item. This process is called **decoding**.



*Figure 3-2. Items of clothing in the infinite wardrobe - each black dot represents an item of clothing.*

# AutoEncoders

- The bottleneck layer in autoencoders serves as the latent space, capturing a compressed representation of the input data.



## Autoencoders

A diagram of the process described by the story is shown in Figure 3-2. You play the part of the **encoder**, moving each item of clothing to a location in the wardrobe. This process is called **encoding**. Brian plays the part of the **decoder**, taking a location in the wardrobe and attempting to re-stitch the item. This process is called **decoding**.

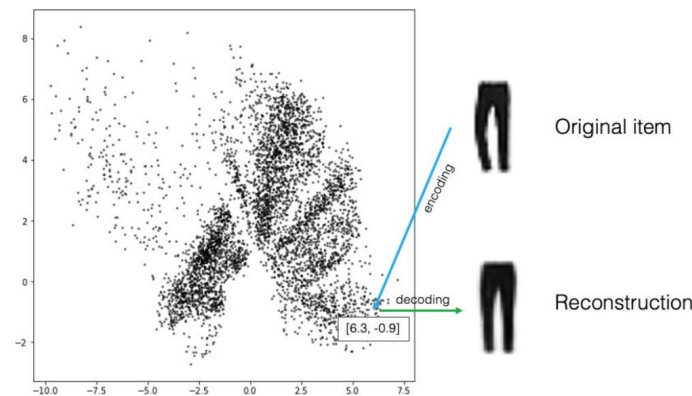
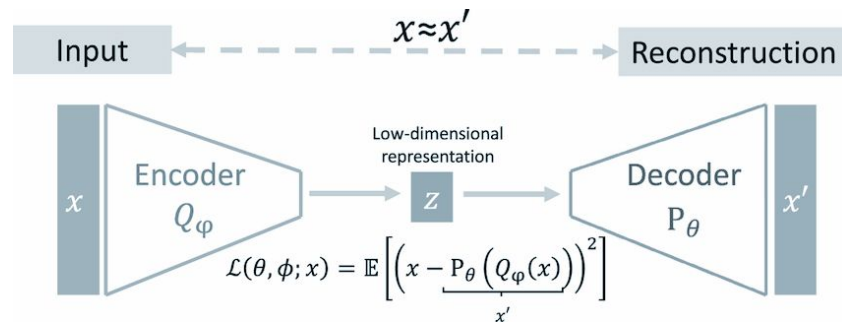


Figure 3-2. Items of clothing in the infinite wardrobe - each black dot represents an item of clothing.

# Training AutoEncoders

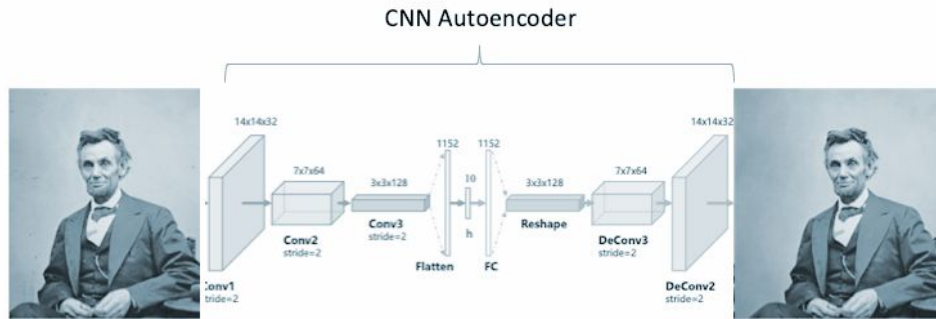
There are 4 fundamental hyperparameters for training an autoencoder:

- **Code/Latent Space Size:** The bottleneck size decides how much the data has to be compressed. This can also act as a regularisation term.
- **Number of layers/nodes:** While a higher depth/number of nodes increases model capacity/complexity, a lower depth is faster to process and avoid overfitting.
- **Reconstruction Loss:** This is the most important one. It defines the character of the AutoEncoder.



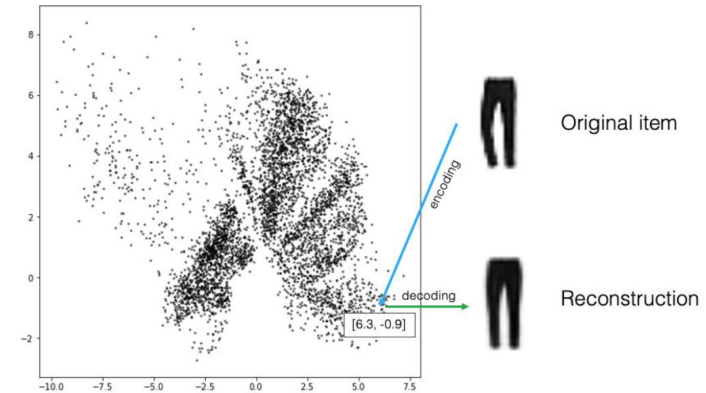
# AutoEncoders

- Autoencoders comes in all types
  - **Convolutional**, (even TCN)
  - LSTM
  - Fully-Connected...



## Autoencoders

A diagram of the process described by the story is shown in **Figure 3-2**. You play the part of the **encoder**, moving each item of clothing to a location in the wardrobe. This process is called **encoding**. Brian plays the part of the **decoder**, taking a location in the wardrobe and attempting to re-stitch the item. This process is called **decoding**.



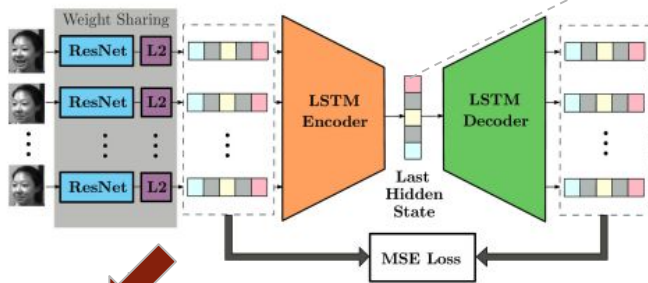
*Figure 3-2. Items of clothing in the infinite wardrobe - each black dot represents an item of clothing.*

Also a good (homework and) project idea,  
on any type of sequence data.

WHAT?

# AutoEncoders

- Autoencoders comes in all types
  - Convolutional
  - LSTM**
  - Fully-Connected



Detecting Driver Drowsiness  
as an Anomaly Using LSTM  
Autoencoders

Gülün Tüfekci<sup>1,2[80]</sup>, Alper Kayabasi<sup>1,2[80]</sup>, Erdem Akagündüz<sup>3</sup>,  
and Ilkay Ulusoy<sup>2</sup>

<sup>1</sup> Research Center, Aselsan Inc., Ankara, Turkey  
{gulin.tufekci, alper.kayabasi}@metu.edu.tr

<sup>2</sup> Department of Electrical and Electronics Engineering,  
Middle East Technical University, Ankara, Turkey  
ilkay@metu.edu.tr

<sup>3</sup> Graduate School of Informatics, Middle East Technical University, Ankara, Turkey  
akagunduz@metu.edu.tr

## Autoencoders

A diagram of the process described by the story is shown in **Figure 3-2**. You play the part of the **encoder**, moving each item of clothing to a location in the wardrobe. This process is called **encoding**. Brian plays the part of the **decoder**, taking a location in the wardrobe and attempting to re-stitch the item. This process is called **decoding**.

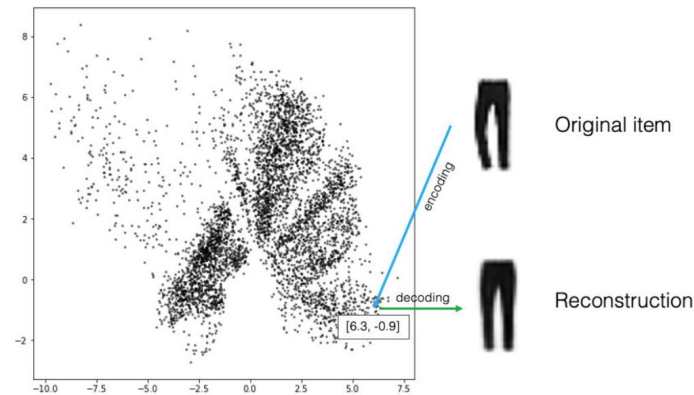
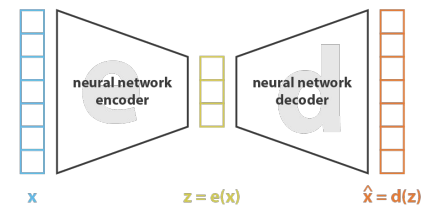
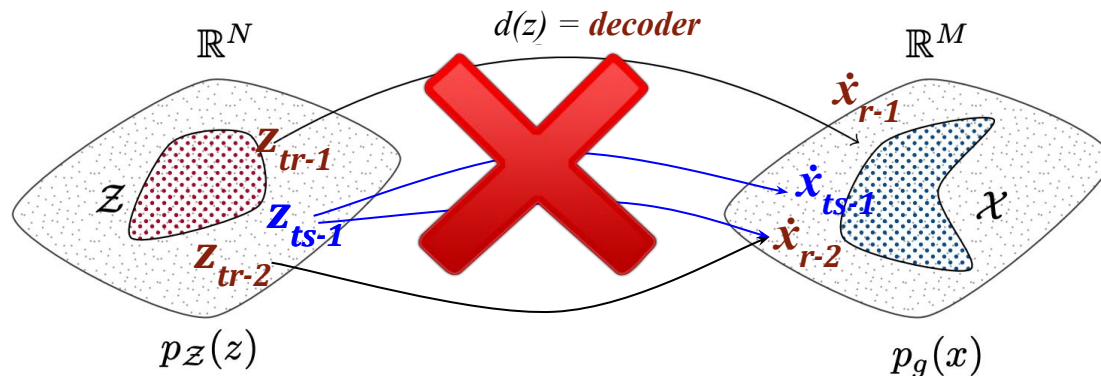


Figure 3-2. Items of clothing in the infinite wardrobe - each black dot represents an item of clothing.

# AE problems



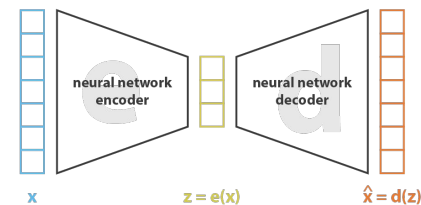
- One of the main limitations of traditional AEs is that they do not necessarily create a continuous latent space.
- This discontinuity makes it difficult for AEs to perform tasks like interpolation or extrapolation, where generating new samples in between or beyond the training data becomes challenging.



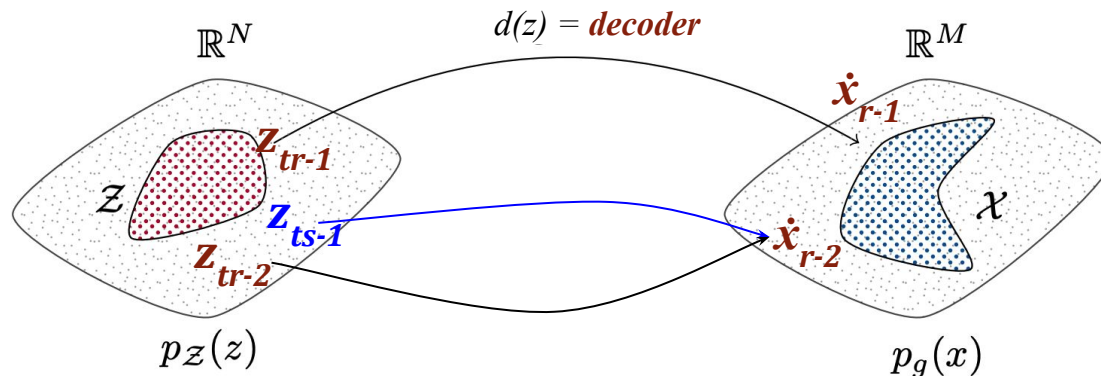
because it  
memorises the  
training  
samples



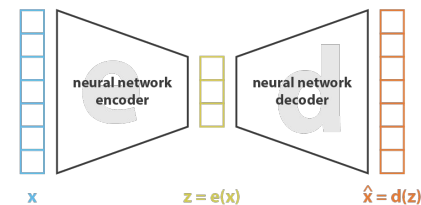
# AE problems



- The fundamental problem with autoencoders, for generation, is that the latent space they convert their inputs to and where their encoded vectors lie, may not be continuous, or allow easy interpolation.
- This is fine if you're just replicating the same images.



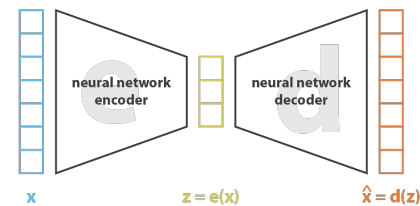
# Latent Space size ?



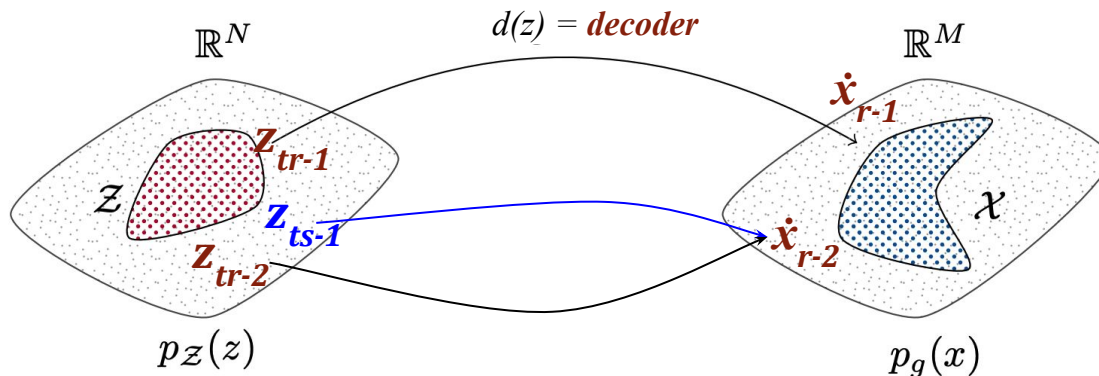
- Increasing the latent space vector  $z$  dimension size ( $n$ ), will improve the replication/reconstruction success, but will affect the continuity problem even worse.
- Increasing the latent space vector  $z$  dimension size ( $n$ ), will degrade the replication/reconstruction performance.

| Input image                                                                       | 2-D latent space                                                                  | 5-D latent space                                                                   | 10-D latent space                                                                   |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|  |  |  |  |

# AE problems

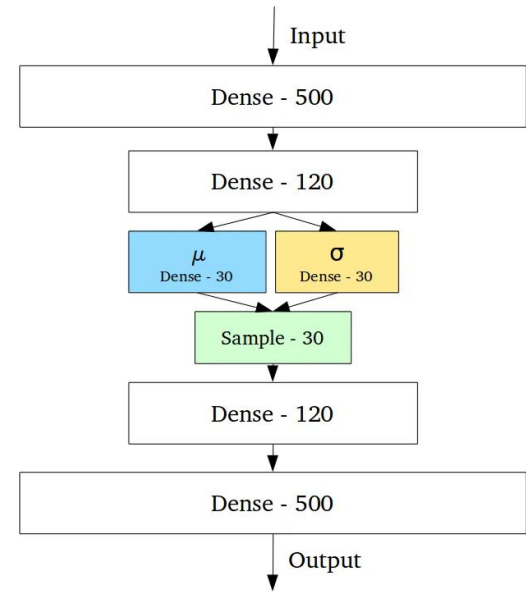


- But when you're building a generative model, you don't want to replicate the same input.
- You want to randomly sample from the latent space, or generate variations on an input image, from a continuous latent space.



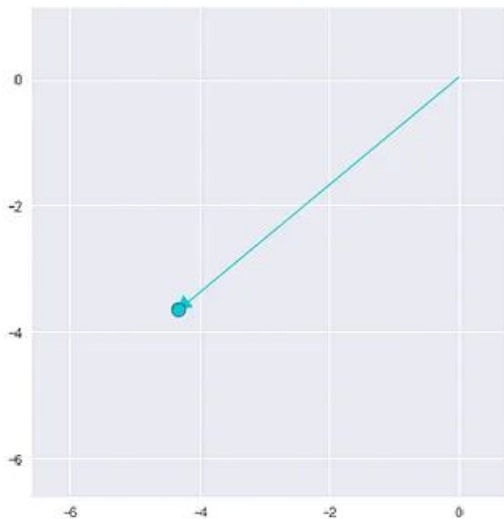
# Variational AutoEncoders (VAEs)

- Variational Autoencoders (VAEs) have one fundamentally unique property that separates them from vanilla autoencoders, and it is this property that makes them so useful for generative modeling:
- their latent spaces are, **by design**, continuous, allowing easy random sampling and interpolation.

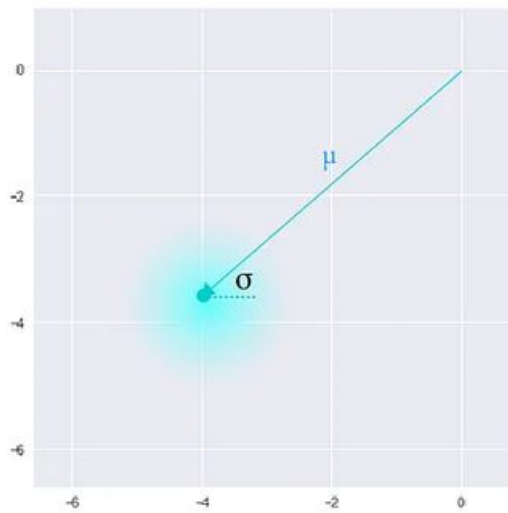


# Variational AutoEncoders (VAEs)

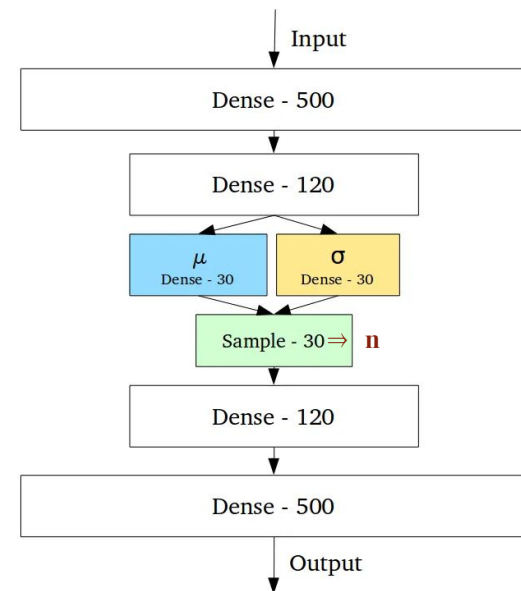
- Instead of mapping the input to a fixed latent vector, they map it to a distribution!



Standard Autoencoder  
(direct encoding coordinates)

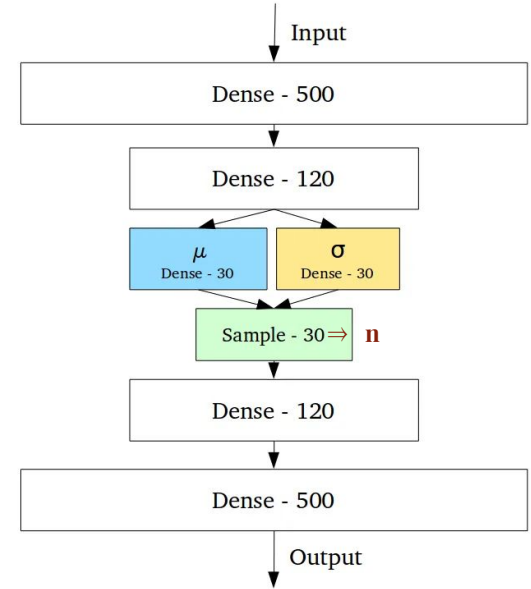


Variational Autoencoder  
( $\mu$  and  $\sigma$  initialize a probability distribution)



# Variational AutoEncoders (VAEs)

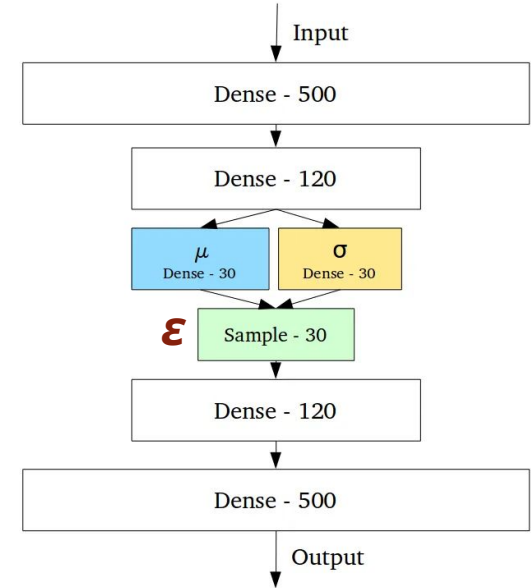
- Instead of mapping the input to a fixed latent vector, they map it to a distribution!
- What is more, VAEs force the latent variables to be Normal distributed.
- It achieves this by making its encoder :
  - *not output an encoding vector of size  $n$  (like AEs),*
  - *rather, outputting two vectors of size  $n$ :*
    - *a vector of means,  $\mu$ ,*
    - *and another vector of standard deviations,  $\sigma$ .*



# Variational AutoEncoders (VAEs)

- To achieve this, VAEs introduce two additional layers in the bottleneck: the mean layer and the standard deviation layer.
- These layers take the output of the previous bottleneck layer and output the mean vector ( $\mu$ ) and the standard deviation vector ( $\sigma$ ) respectively.
- Specifically, we sample a vector  $\epsilon$  from a standard normal distribution (zero-mean, unit var), and sample  $\mathbf{z}$ , accordingly.

$$z = \mu + \sigma * \varepsilon$$

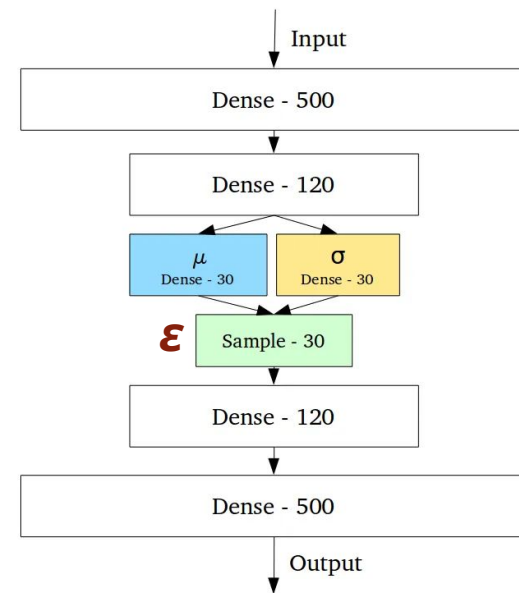


# Variational AutoEncoders (VAEs)

- Here,  $\epsilon$  serves as a source of randomness and allows us to sample different points from the latent space during training or generation.
- Ok. Perfect. Forward run is stochastic! Great! But:

**How is this random operation handled during training? Can you backpropagate a layer that creates a random variable?**

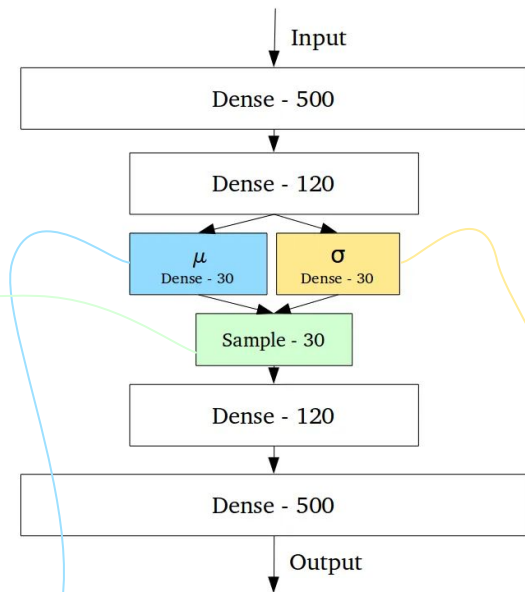
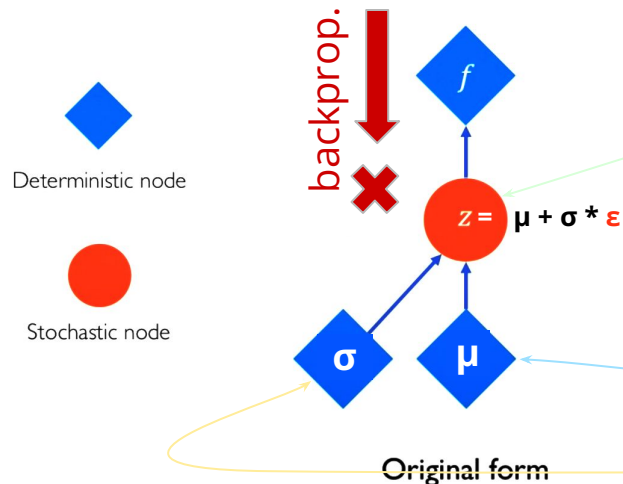
$$\mathbf{z} = \mu + \sigma * \epsilon$$





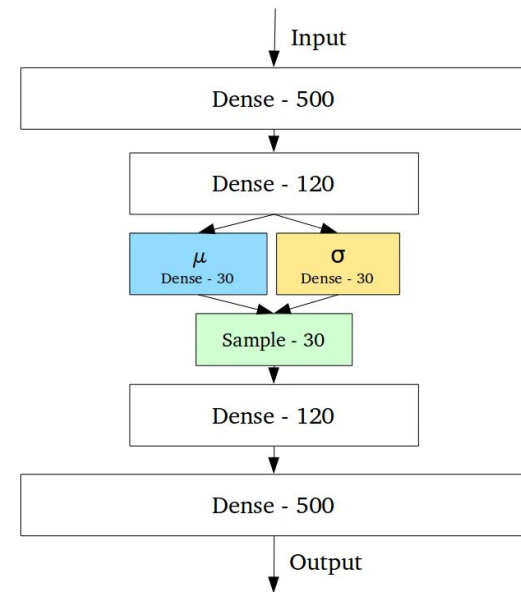
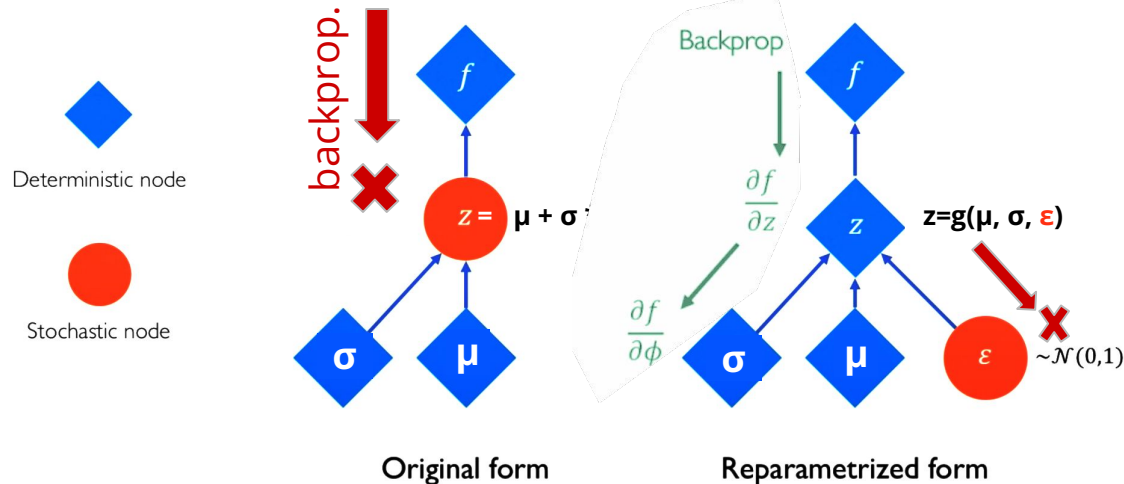
# Reparameterization Trick

- The random sampling operation in VAEs, where  $\epsilon$  is drawn from a standard normal distribution, introduces a challenge for backpropagation since it involves a non-differentiable operation.



# Reparameterization Trick

- The random sampling operation in VAEs, where  $\epsilon$  is drawn from a standard normal distribution, introduces a challenge for backpropagation since it involves a non-differentiable operation.



# V-AutoEncoder Loss function

- In a VAE, the loss function plays a crucial role in training the model and optimizing the reconstruction process.
- The choice of the loss function depends on the specific objectives and characteristics of the autoencoder.
- There can be many features that shape the loss character of an AE
  - Reconstruction (main)
  - Regularization (main - VAE)
  - Characterized loss (optional): Sparsity, Variational loss, other problem dependent

# Reconstruction Loss

- The primary objective of an autoencoder is to reconstruct the input data from the compressed latent space.
- The reconstruction loss measures the discrepancy between the original input and the reconstructed output.
- Commonly used reconstruction loss functions depend on the input data and include:
  - mean squared error (continuous data), binary cross-entropy (binary data),

# Regularization Loss

- VAEs can be prone to overfitting, especially when the model capacity is high or the dataset is limited.
- Regularization techniques, such as L1 or L2 regularization, drop-out etc, can be incorporated into the loss function to discourage complex or redundant representations.
- Regularization loss plays a vital role in controlling the behavior of an autoencoder and ensuring that the learned representations possess desirable properties.
- The specific choice and application of regularization techniques depend on the characteristics of the data, the desired properties of the representations, and the overall training objectives.

# Characterized Loss

- A “characterized loss” component can be introduced to go beyond traditional objectives like reconstruction or regularization, addressing specific requirements or goals of the task at hand.
- Such as:
  - Sparsity Loss: Only a small subset of the latent variables or activations should be active or non-zero.
    - $\lambda * \sum |z|$
  - Diversity Loss: Generation of diverse outputs by penalizing models that generate repetitive or similar samples.
    - $(1/(n*(n-1))) * \sum_{i=1}^n \sum_{j=1, j \neq i}^n d(x_i, x_j)$
  - Consistency Loss: encourages the model to produce consistent predictions across augmented versions of the same sample, improving generalization.

# Unsupervised Clustering of Seismic Signals Using Deep Convolutional Autoencoders

S. Mostafa Mousavi<sup>1</sup>, Weiqiang Zhu<sup>2</sup>, William Ellsworth<sup>3</sup>, and Gregory Beroza<sup>4</sup>

**This is a very good project idea.  
The data will be provided!**

## Characterized Loss: Clustering

- A clustering layer, connected to the AE's bottleneck, assigns the hidden features of each sample to a cluster.
- The clustering loss component  $L_c$  is specifically designed to enhance the clustering behavior of the learned latent representations.
- The input is data is seismic waveforms, that reach a seismic station.

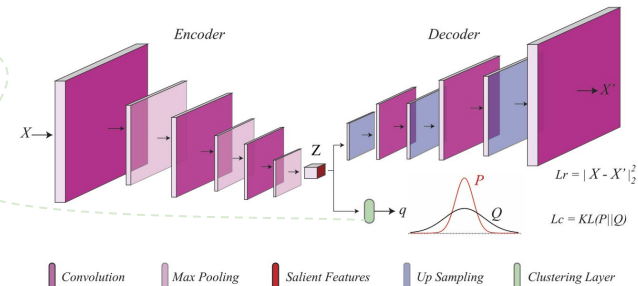


Fig. 1. Network architecture. The encoder and decoder are composed of fully convolutional layers followed by max-pooling and up-sampling layers, respectively. Reconstruction loss ( $L_r$ ) and the clustering loss ( $L_c$ ) are given in the figure.

$$L = (1 - \lambda)L_r + \lambda L_c \quad (1)$$

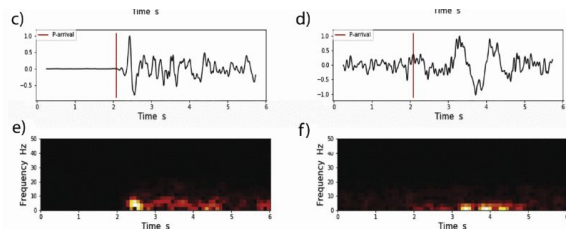
$$q_{ij} = \frac{(1 + \|z_i - \mu_j\|^2)^{-1}}{\sum_j (1 + \|z_i - \mu_j\|^2)^{-1}} \quad (2)$$

The membership probabilities are used to compute an auxiliary target distribution,  $p_{ij}$

$$p_{ij} = \frac{q_{ij}^2 / \sum_i q_{ij}}{\sum_j (q_{ij}^2 / \sum_i q_{ij})} \quad (3)$$

and clustering is performed by minimizing the Kullback-Leibler (KL) divergence between the soft assignments,  $q_{ij}$ , and the target distribution,  $p_{ij}$

$$L_c = KL(P \parallel Q) = \sum_i \sum_j p_{ij} \log \left( \frac{p_{ij}}{q_{ij}} \right) \quad (4)$$



# Characterized Loss: Clustering

- The clustering is done in two steps.
  - In the initial step, they train by setting  $\lambda=0$  (i.e. only reconstruction)
  - In the next step, the learned features are used to initialize the cluster centroids ( $\mu_j$ ) in the feature space using k-means. Following, cluster assignment and feature learning are jointly performed ( $\lambda=0.1$ )
- Or, just for fun (and for a good project) regions can be manually labeled for **regions**, and the second step could be supervised! That would be a mix of unsupervised and supervised learning!

## Unsupervised Clustering of Seismic Signals Using Deep Convolutional Autoencoders

S. Mostafa Mousavi<sup>1</sup>, Weiqiang Zhu<sup>2</sup>, William Ellsworth<sup>3</sup>, and Gregory Beroza<sup>4</sup>

**This is a very good project idea.  
The data will be provided!**

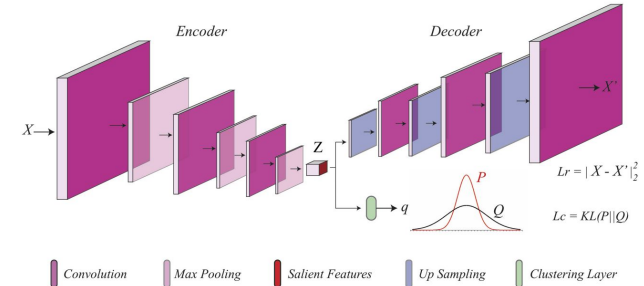


Fig. 1. Network architecture. The encoder and decoder are composed of fully convolutional layers followed by max-pooling and up-sampling layers, respectively. Reconstruction loss ( $L_r$ ) and the clustering loss ( $L_c$ ) are given in the figure.

$$L = (1 - \lambda)L_r + \lambda L_c \quad (1)$$

$$q_{ij} = \frac{(1 + \|z_i - \mu_j\|^2)^{-1}}{\sum_j (1 + \|z_i - \mu_j\|^2)^{-1}} \quad (2)$$

The membership probabilities are used to compute an auxiliary target distribution,  $p_{ij}$

$$p_{ij} = \frac{q_{ij}^2 / \sum_i q_{ij}}{\sum_j (q_{ij}^2 / \sum_i q_{ij})} \quad (3)$$

and clustering is performed by minimizing the Kullback-Leibler (KL) divergence between the soft assignments,  $q_{ij}$ , and the target distribution,  $p_{ij}$

$$L_c = KL(P \parallel Q) = \sum_i \sum_j p_{ij} \log \left( \frac{p_{ij}}{q_{ij}} \right) \quad (4)$$



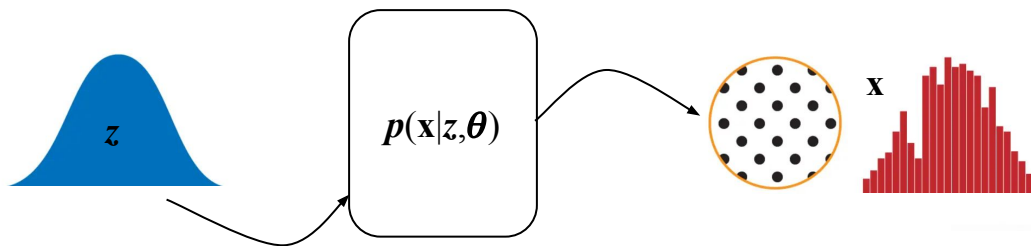
# VAE Loss as ELBO

- To train a VAE, we need to maximize the likelihood of the observed data 'x' given the model parameters ' $\theta$ '. However, directly computing  $p(x | z, \theta)$  is not feasible because we do not have access to the “true” latent variables 'z', and the encoding process is not perfectly reversible.
- Instead, VAEs use the Evidence Lower Bound (ELBO) as an approximation to the log-likelihood. The ELBO involves two terms: the expected log-likelihood of the data given the latent space (decoder part) and the negative KL divergence between the approximate posterior (encoded distribution) and the prior distribution over the latent space.

$$\mathcal{L}(\phi, \theta, x) = \boxed{(\text{reconstruction loss})} + (\text{regularization term})$$

# VAE Loss as ELBO

- By maximizing the ELBO, the VAE effectively encourages the latent space to be structured and follow the prior distribution (usually a standard normal distribution), and it encourages the decoder to generate data that is similar to the observed data.
- The inability to directly calculate  $p(\mathbf{x} | \mathbf{z}, \theta)$  due to the non-invertibility of the encoder is the reason why VAEs use ELBO for training instead of maximum likelihood.



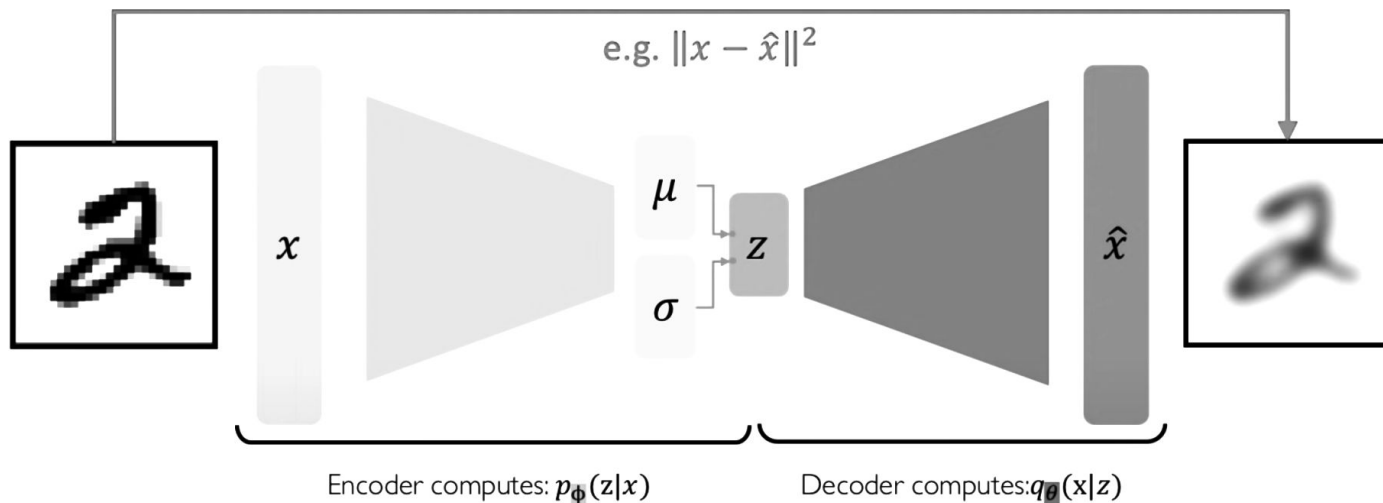
# VAE Loss

- So we know how to backprop a VAE. Ready for training then!
- During training, VAEs has two optimization goals:
  - minimize the reconstruction error (similar to traditional autoencoders)
  - and to ensure that the generated latent vectors follow the desired probability distribution.
- The first component is the reconstruction loss and it is trivial.

$$\mathcal{L}(\phi, \theta, x) = \boxed{\text{(reconstruction loss)}} + \text{(regularization term)}$$

# VAE Reconstruction Loss

- Reconstruction loss is straightforward.



$$\mathcal{L}(\phi, \theta, x) = \boxed{\text{(reconstruction loss)}} + \text{(regularization term)}$$

# VAE Regularization Loss

- So we know how to backprop a VAE. Ready for training then!
- During training, VAEs has two optimization goals:
  - minimize the reconstruction error (similar to traditional autoencoders)
  - and to ensure that the generated latent vectors follow the desired probability distribution.
- The second is done by including a regularization term in the loss function called the Kullback-Leibler (KL) divergence.
  - The KL divergence measures how different the distribution of the generated latent vectors is from the desired distribution (in this case, a standard normal distribution).

$$\mathcal{L}(\phi, \theta, x) = (\text{reconstruction loss}) + \boxed{(\text{regularization term})}$$

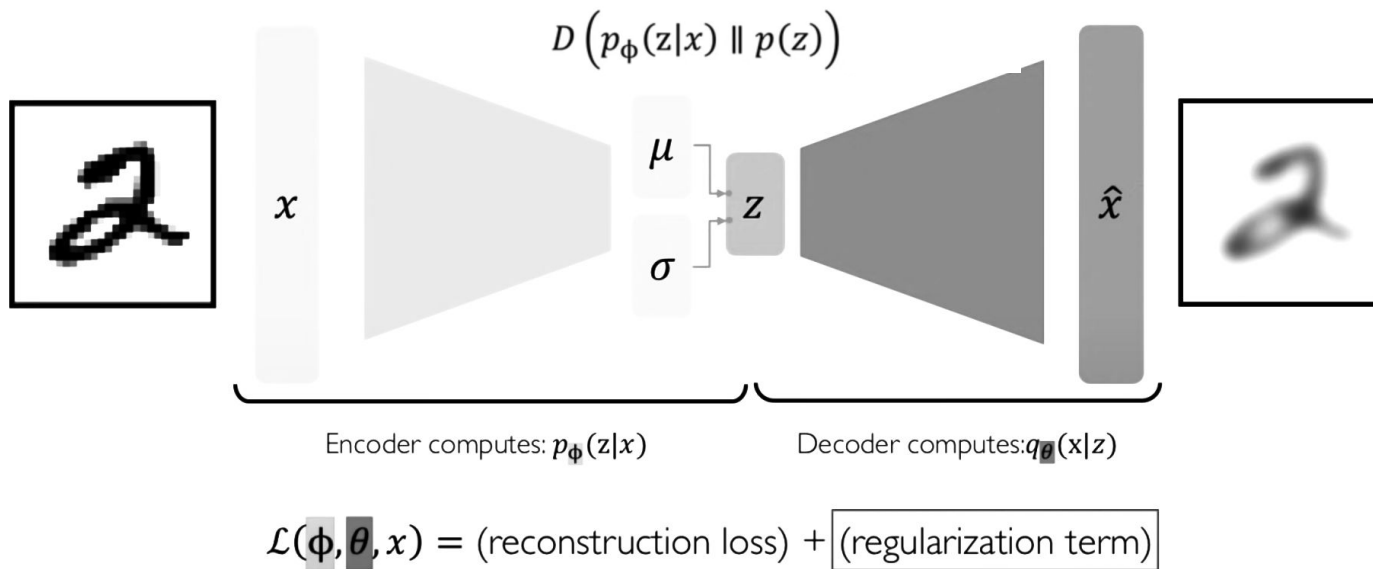
# Kullback-Leibler (KL) Divergence

- The KL divergence is a measure of the difference between two probability distributions, P and Q.
- It is defined as the expected value of the logarithmic difference between P and Q, where the expectation is taken with respect to P. The KL divergence is denoted as  $D(P || Q)$ .

$$D_{KL}(P||Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

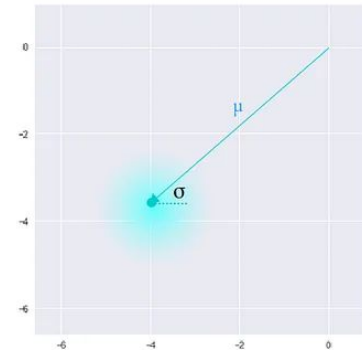
# VAE Regularization Loss

- Regularization loss shapes the distributions.

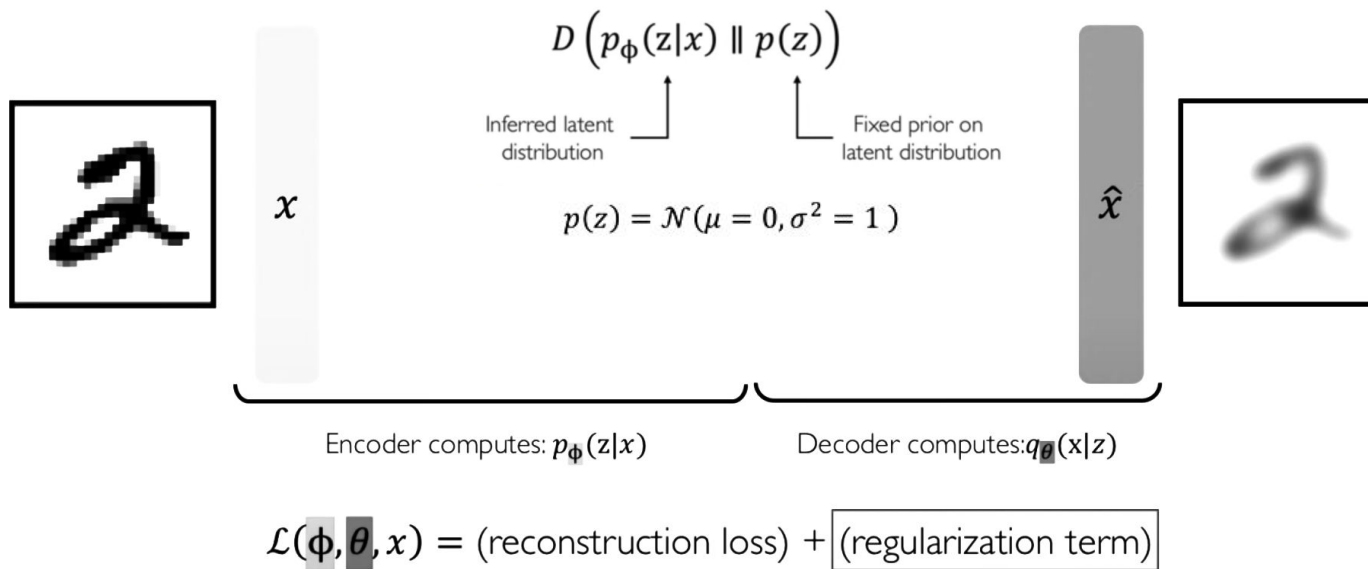


# VAE Regularization Loss

- Regularization loss shapes the distributions.



Variational Autoencoder  
( $\mu$  and  $\sigma$  initialize a probability distribution)





# VAE Regularization Loss

- Regularization loss shapes the distributions.

$$D \left( p_{\phi}(z|x) \parallel p(z) \right)$$

Inferred latent  
distribution

↑

↑

Fixed prior on  
latent distribution

$$p(z) = \mathcal{N}(\mu = 0, \sigma^2 = 1)$$

- The KL divergence measures how different the distribution of the generated latent vectors is from the desired distribution (in this case, a standard normal distribution).

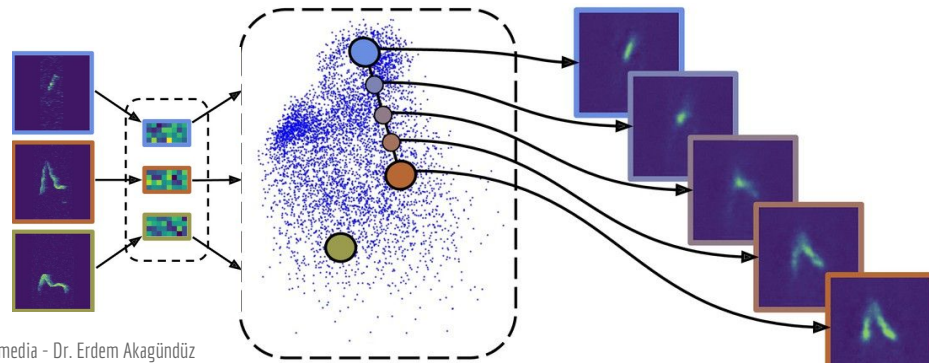
# VAE → Continuity

- What do we want to achieve?

- Continuity:** points close to each other in the latent space → similar content.  
*This property ensures that neighboring points in the latent space correspond to similar variations or features in the input data.*
  - Example:

*Point A might represent a digit "1," while point B might represent a digit "2." Since they are close in the latent space, the decoded images will display gradual changes from "1" to "2," capturing the continuity of the digit variations.*

figure from <https://doi.org/10.7554/eLife.67855>



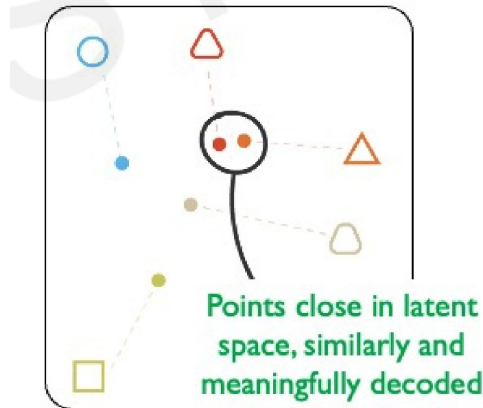
# VAE $\rightarrow$ Continuity

- What do we want to achieve?

- **Continuity:** points close to each other in the latent space  $\rightarrow$  similar content.  
*This property ensures that neighboring points in the latent space correspond to similar variations or features in the input data.*

- Example:

*Point A might represent a digit "1," while point B might represent a digit "2." Since they are close in the latent space, the decoded images will display gradual changes from "1" to "2," capturing the continuity of the digit variations.*

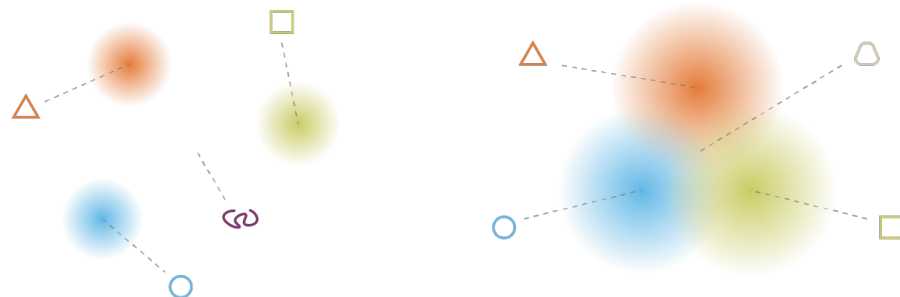


# VAE → Completeness

- What do we want to achieve?

- Completeness:** sampling arbitrarily in the latent space → always meaningful content!  
The latent space always captures meaningful and diverse representations of the input data. (like “generalized continuity”)
  - Example:

*By sampling different latent vectors from the space, we can generate diverse faces, each with unique combinations of these attributes. This demonstrates the completeness of the latent space in generating meaningful and varied content.*



what can happen without regularisation



what we want to obtain with regularisation

# VAE → what else?

- What else in a latent space and for what other purpose can we manipulate?
  - *Clustering/Classification*: By grouping similar data points together in the latent space, you can discover patterns or clusters within the data. Additionally, by training a classifier on top of the latent space, you can perform classification tasks directly in the latent space.

## Unsupervised Clustering of Seismic Signals Using Deep Convolutional Autoencoders

S. Mostafa Mousavi<sup>✉</sup>, Weiqiang Zhu<sup>✉</sup>, William Ellsworth<sup>✉</sup>, and Gregory Beroza

**This is a very good project idea.**  
**The data will be provided!**

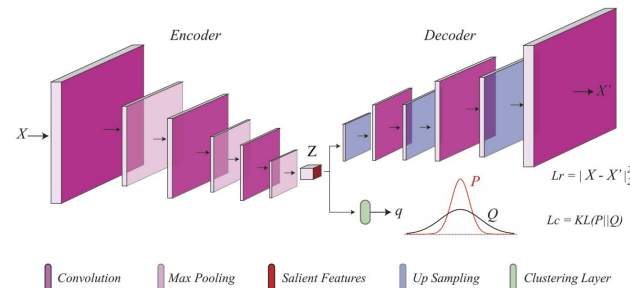


Fig. 1. Network architecture. The encoder and decoder are composed of fully convolutional layers followed by max-pooling and up-sampling layers, respectively. Reconstruction loss ( $L_r$ ) and the clustering loss ( $L_c$ ) are given in the figure.

$$L = (1 - \lambda)L_r + \lambda L_c \quad (1)$$

$$q_{ij} = \frac{(1 + \|z_i - \mu_j\|^2)^{-1}}{\sum_j (1 + \|z_i - \mu_j\|^2)^{-1}} \quad (2)$$

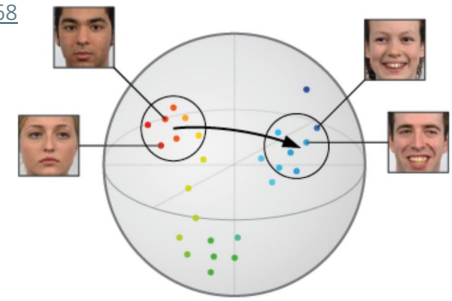
The membership probabilities are used to compute an auxiliary target distribution,  $p_{ij}$

$$p_{ij} = \frac{q_{ij}^2 / \sum_i q_{ij}}{\sum_j (q_{ij}^2 / \sum_i q_{ij})} \quad (3)$$

and clustering is performed by minimizing the Kullback-Leibler (KL) divergence between the soft assignments,  $q_{ij}$ , and the target distribution,  $p_{ij}$

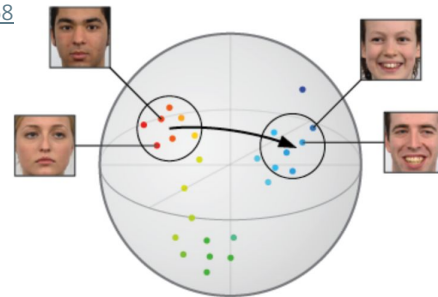
$$L_c = KL(P \parallel Q) = \sum_i \sum_j p_{ij} \log \left( \frac{p_{ij}}{q_{ij}} \right) \quad (4)$$

# VAE $\rightarrow$ what else?



- *Interpolation*: The latents space can generate intermediate outputs that **smoothly** transition between the corresponding reconstructions (like smoothness in continuity).
- For Linear interpolation in the latent space, you don't need to make any specific changes to the loss function itself. Linear interpolation is a property that emerges naturally from the training process of a well-designed Variational Autoencoder (VAE) with appropriate loss functions.
- However, for example, [White, 2016] proposes spherical interpolation (instead of linear).

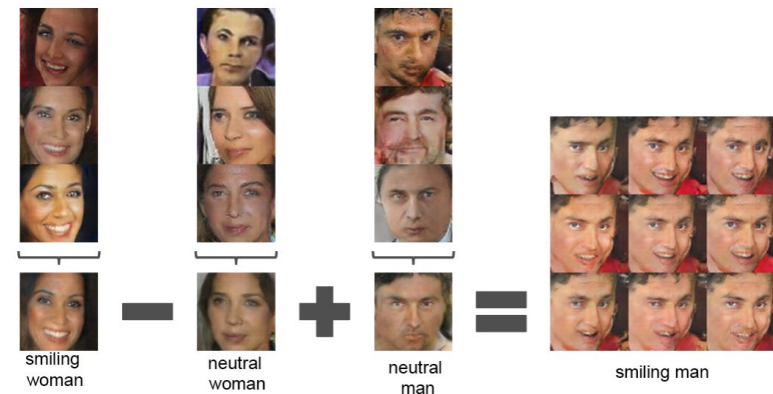
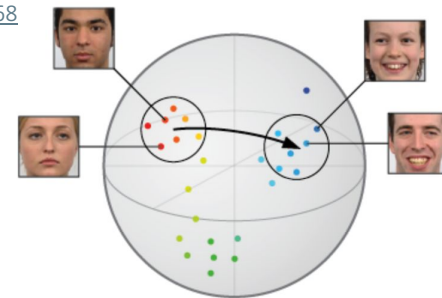
# VAE $\rightarrow$ what else?



- *Spherical Interpolation?*
- You can do this:
  - By post processing the latent space
  - Or add Spherical Normalization Layer:
    - You can add a layer at the end of the encoder that performs spherical normalization.
    - This layer would take the output of the encoder, normalize the latent vectors to have unit length, and enforce that they lie on the surface of a hypersphere.
    - This normalization can be achieved by dividing each latent vector by its Euclidean norm. By incorporating this layer, you encourage the latent vectors to be distributed on a hypersphere, which facilitates spherical interpolation during inference.
    - (not the loss function this time)

# VAE → what else?

- *Arithmetic Operations*: Latent spaces can sometimes exhibit interesting linear relationships between attributes or features of the data. For example, in a face-related VAE, one latent dimension might represent a certain expression.
- By manipulating the value of this dimension, such as adding or subtracting a certain amount, you can control the age of the generated face.


figure from <https://arxiv.org/abs/1511.06434>

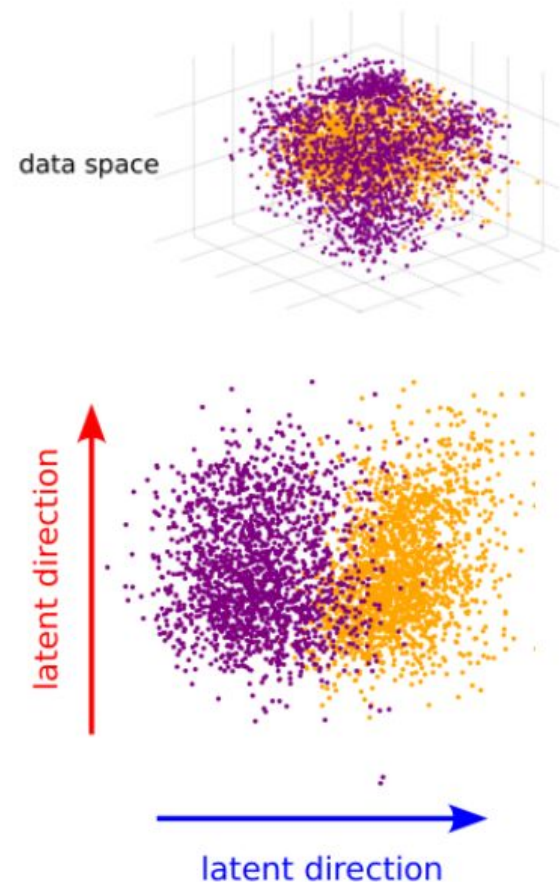


# VAE → what else?

- *Arithmetic Operations*
- You typically do not need specific manipulations in the loss function for this, **BUT** you can encourage (for example) *linear* operations in the latent space by incorporating specific manipulations in the VAE architecture or loss function, by
  - adding a regularization term to the loss function that encourages linearity in the latent space and penalizes deviations from linearity.
  - including additional layers or constraints that enforce linear transformations or encourage linearity between specific dimensions in the latent space.
  - Or by **disentangling** (?) the latent space.

# Disentanglement

- Disentanglement in the latent space refers to the ability ***capturing independent factors of variation or attributes in a way that each dimension in the latent space corresponds to a specific and interpretable aspect of the data.***
- This property allows for more explicit control and manipulation of specific attributes or factors.
- It is like doing a PCA in the latent space!



# Disentanglement

- They create an intermediate latent space  $y$  (out of the original latent space  $z$ ), which can disentangle certain attributes such as gender, as shown in the figure.
- They learn the intermediate latents space  $y$ !

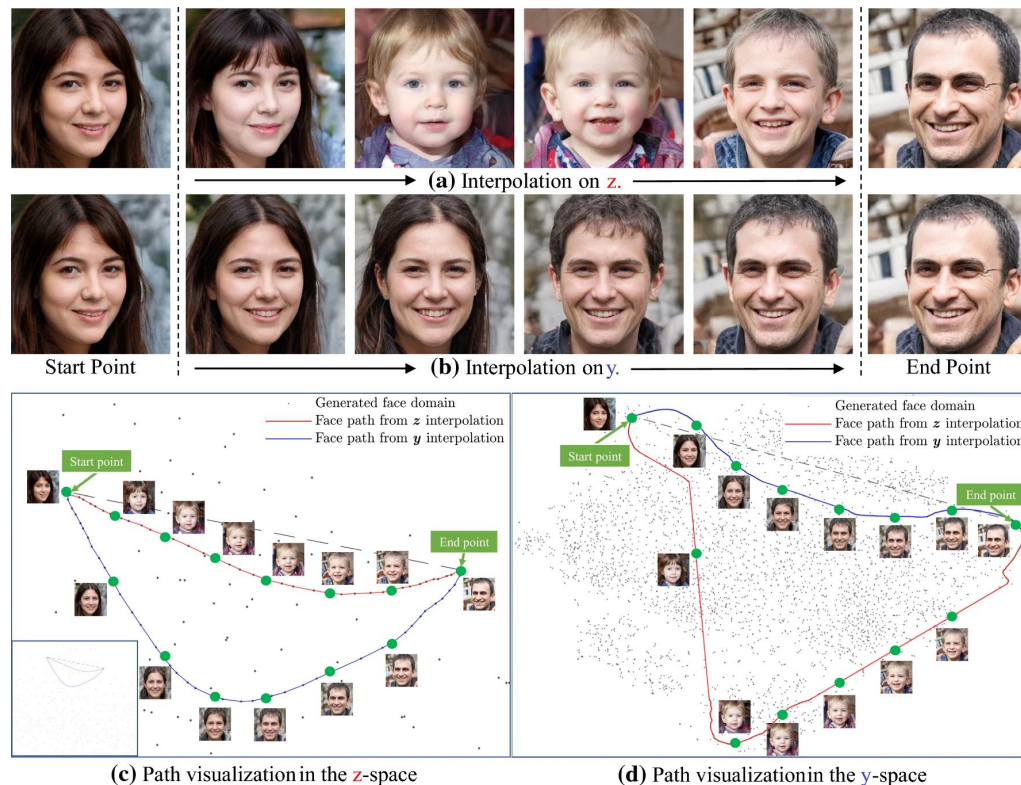
Published: 22 March 2022

## Disentangled Inference for GANs With Latently Invertible Autoencoder

Jiapeng Zhu, Deli Zhao, Bo Zhang & Bolei Zhou ✉

*International Journal of Computer Vision* 130, 1259–1276 (2022) | [Cite this article](#)

725 Accesses | 6 Citations | 1 Altmetric | [Metrics](#)



Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot,  
Matthew Botvinick, Shakir Mohamed, Alexander Lerchner  
Google DeepMind  
{irinah, lmatthey, arkap, cpburgess, glorotx,  
botvinick, shakir, lerchner}@google.com

## ABSTRACT

Learning an interpretable factorised representation of the independent data generative factors of the world without supervision is an important precursor for the development of artificial intelligence that is able to learn and reason in the same way that humans do. We introduce  $\beta$ -VAE, a new state-of-the-art framework for automated discovery of interpretable factorised latent representations from raw image data in a completely unsupervised manner. Our approach is a modification of the variational autoencoder (VAE) framework. We introduce an adjustable hyperparameter  $\beta$  that balances latent channel capacity and independence constraints with reconstruction accuracy. We demonstrate that  $\beta$ -VAE with appropriately tuned  $\beta > 1$  qualitatively outperforms VAE ( $\beta = 1$ ), as well as state of the art unsupervised (InfoGAN) and semi-supervised (DC-IGN) approaches to disentangled factor learning on a variety of datasets (*celebA*, *faces* and *chairs*). Furthermore, we devise a protocol to quantitatively compare the degree of disentanglement learnt by different models, and show that our approach also significantly outperforms all baselines quantitatively. Unlike InfoGAN,  $\beta$ -VAE is stable to train, makes few assumptions about the data and relies on tuning a single hyperparameter  $\beta$ , which can be directly optimised through a hyperparameter search using weakly labelled data or through heuristic visual inspection for purely unsupervised data.

# Disentanglement in VAEs

- There have been several published papers that explore different techniques and modifications to achieve disentanglement in the latent space of VAEs, such as:
  - " $\beta$ -VAE" by Higgins et al. (2017): This paper introduces the concept of  $\beta$ -VAE, which controls the disentanglement and linearity of the learned representations by adjusting a hyperparameter. By manipulating the  $\beta$  value, they demonstrate the ability to encourage linear variations in the latent space.
  - $\beta$ -VAE introduces a manipulation in the loss function to encourage certain properties in the latent space, including disentanglement and potentially linearity.



# $\beta$ -VAE

## VAE vs $\beta$ -VAE

- The regularization term in beta-VAE is multiplied by a factor  $\mathcal{F}(\theta, \phi, \beta; \mathbf{x}, \mathbf{z}) = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{z})] - \beta (D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z})) - \epsilon)$  (3)
- ...and an additional term that penalizes the correlation between the latent variables.
- $\beta$ -VAE also uses a linear classifier to learn a target factor on the latent variables.

Figure 5: Schematic of the proposed disentanglement metric: over a batch of  $L$  samples, each pair of images has a fixed value for one target generative factor  $y$  (here  $y = scale$ ) and differs on all others. A linear classifier is then trained to identify the target factor using the average pairwise difference  $\mathbf{z}_{diff}^b$  in the latent space over  $L$  samples.

<https://openreview.net/pdf?id=Sy2fzU9gl>

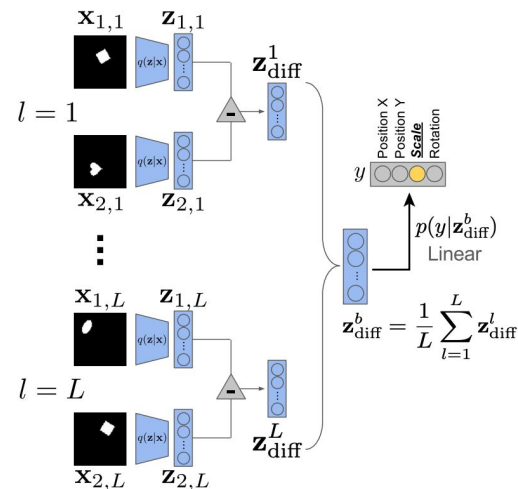
| Feature                        | Vanilla VAE                         | Beta-VAE                                                  |
|--------------------------------|-------------------------------------|-----------------------------------------------------------|
| Loss function                  | Reconstruction loss + KL divergence | Reconstruction loss + KL divergence + correlation penalty |
| Difficulty of training         | Easier                              | More difficult                                            |
| Sensitivity to hyperparameters | Less sensitive                      | More sensitive                                            |

## $\beta$ -VAE: LEARNING BASIC VISUAL CONCEPTS WITH A CONSTRAINED VARIATIONAL FRAMEWORK

Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, Alexander Lerchner  
Google DeepMind  
{irinah, lmatthey, arkap, cpburgess, glorotx, botvinick, shakir, lerchner}@google.com

### ABSTRACT

Learning an interpretable factorised representation of the independent data generative factors of the world without supervision is an important precursor for the development of artificial intelligence that is able to learn and reason in the same way that humans do. We introduce  $\beta$ -VAE, a new state-of-the-art framework for automated discovery of interpretable factorised latent representations from raw image data in a completely unsupervised manner. Our approach is a modification of the variational autoencoder (VAE) framework. We introduce an adjustable hyperparameter  $\beta$  that balances latent channel capacity and independence constraints with reconstruction accuracy. We demonstrate that  $\beta$ -VAE with appropriately tuned  $\beta > 1$  qualitatively outperforms VAE ( $\beta = 1$ ), as well as state of the art unsupervised (InfoGAN) and semi-supervised (DC-IGN) approaches to disentangled factor learning on a variety of datasets (*celebA*, *faces* and *chairs*). Furthermore, we devise a protocol to quantitatively compare the degree of disentanglement learnt by different models, and show that our approach also significantly outperforms all baselines quantitatively. Unlike InfoGAN,  $\beta$ -VAE is stable to train, makes few assumptions about the data and relies on tuning a single hyperparameter  $\beta$ , which can be directly optimised through a hyperparameter search using weakly labelled data or through heuristic visual inspection for purely unsupervised data.



# Disentanglement in VAEs

- "Disentangling by Factorising" by Kim and Mnih (2018)

This paper proposes FactorVAE, an extension of the  $\beta$ -VAE framework that focuses on disentanglement.

- FactorVAE includes a disentanglement metric that encourages linearity in the latent space by explicitly measuring independence between factors of variation.
- FactorVAE introduces a specific metric called the Total Correlation (TC) term as part of the regularization in the loss function. The TC term aims to measure the degree of dependence between different factors of variation captured in the latent space. By minimizing the TC term, FactorVAE encourages the latent space to be disentangled, with separate dimensions representing independent factors.

Hyunjik Kim<sup>1,2</sup> Andriy Mnih<sup>1</sup>

## Abstract

We define and address the problem of unsupervised learning of disentangled representations on data generated from independent factors of variation. We propose FactorVAE, a method that disentangles by encouraging the distribution of representations to be factorial and hence independent across the dimensions. We show that it improves upon  $\beta$ -VAE by providing a better trade-off between disentanglement and reconstruction quality. Moreover, we highlight the problems of a commonly used disentanglement metric and introduce a new metric that does not suffer from them.

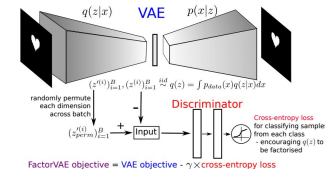


Figure 1. Architecture of FactorVAE, a Variational Autoencoder (VAE) that encourages the code distribution to be factorial. The top row is a VAE with convolutional encoder and decoder, and the bottom row is an MLP classifier, the discriminator, that distinguishes whether the input was drawn from the marginal code distribution or the product of its marginals.

$$\begin{aligned}
 TC(z) &= KL(q(z)||\bar{q}(z)) = \mathbb{E}_{q(z)} \left[ \log \frac{q(z)}{\bar{q}(z)} \right] \\
 &\approx \mathbb{E}_{q(z)} \left[ \log \frac{D(z)}{1 - D(z)} \right].
 \end{aligned} \tag{3}$$

# What will we do next week?

- Starting with next week...
  - *Normalizing Flows, Flow-based Deep Generative Models*
- Following weeks
  - *Other things...*

# Additional Reading & References

- <https://towardsdatascience.com/gaussian-mixture-models-and-expectation-maximization-a-full-explanation-50fa94111ddd>
- <https://www.v7labs.com/blog/autoencoders-guide>
- <https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf>
- <https://towardsdatascience.com/reparameterization-trick-126062cfd3c3>
- <https://youtu.be/rZufA635dq4?t=1091>
- [http://introtodeeplearning.com/slides/6S191\\_MIT\\_DeepLearning\\_L4.pdf](http://introtodeeplearning.com/slides/6S191_MIT_DeepLearning_L4.pdf)
- <https://towardsdatascience.com/understanding-variational-autoencoders-vaes-f70510919f73>
- <https://arxiv.org/abs/1609.04468>
- <https://lilianweng.github.io/posts/2018-08-12-vae/>
- <https://arxiv.org/pdf/2007.03898.pdf>